

PROJECT PITCH PACK

RAZORPAY AI PAYMENT RECOVERY

5-MINUTE PITCH & PROJECT DOCUMENTATION

An evidence-based guide to presenting the implemented webhook-to-recovery workflow, ML risk scoring, recovery advice, audit trail, and dashboard.

Prepared from the current repository implementation

Scope: local full-stack project • September 2026

Important: no live-business impact, deployment, or unverified model claims are stated in this document.

Razorpay AI Payment Recovery • Pitch documentation

1. The pitch

One-line project pitch: Razorpay AI Payment Recovery turns a signed failed-payment webhook into an explainable, tracked recovery path—risk score, cause-aware recommendation, optional payment link, and outcome record—so merchants can act on payment failures instead of merely seeing them.

2. Problem statement

A failed payment is often the end of the visible checkout journey for a merchant. The merchant may know that a payment failed, but not what response is appropriate: wait and retry, ask the customer to authenticate, offer another method, issue a new payment link, or involve support. Repeating the same action for every failure can be ineffective and creates poor customer experience.

3. Why it matters to merchants

- Each failed checkout can become lost revenue unless there is a clear, timely next step.
- Error messages alone are technical and fragmented; operational teams need an interpretable recovery recommendation and a record of what happened.
- Merchants need to distinguish a potentially temporary issue (for example, a timeout) from a customer-action issue (such as invalid card details or insufficient funds).
- They also need accountability: event, insight, strategy, action, and final outcome should be traceable.

4. Our implemented solution

The project is a local, full-stack payment-recovery workflow. It accepts Razorpay-style payment webhooks, verifies the HMAC signature, stores the event, analyses failed payments, records a recovery recommendation, optionally creates a Razorpay Payment Link when credentials are configured, and marks the recovery successful when a uniquely matching captured-payment webhook arrives.

Scope honesty: SMS, email, and customer-support delivery are recorded as pending channel actions; the code does not send them. A retry is a recommendation with a stored strategy—not an automatic charge retry. Without Razorpay credentials, a clearly labelled mock payment-link identifier is stored and remains pending. Groq advice is used only when configured; otherwise deterministic rules are used.

5. End-to-end workflow

```
Razorpay payment.failed webhook
  ↓ (raw body + X-Razorpay-Signature verified with HMAC-SHA256)
Spring Boot webhook endpoint → payment_events (idempotent event_id)
  ↓
SystemInsightService → ML FastAPI/scikit-learn score OR rules fallback
  ↓                               + error-text root-cause categorisation
AiAgentService → Groq structured recovery advisor OR deterministic rules
  ↓
RecoveryActionService → record action; create Razorpay Payment Link only for
                        payment_link recommendations and configured credentials
  ↓
recovery_outcomes = PENDING
  ↓
Razorpay payment.captured webhook → uniquely match by replacement payment ID,
or matching order ID + amount → mark SUCCESS, amount and recovery time
```

6. Features actually implemented

Capability	Implemented behavior
Webhook intake	POST /api/webhooks/payment accepts the raw JSON body and Razorpay signature header.
Signature protection	HMAC-SHA256 is calculated on the exact raw request body and compared in constant time. Unsigned requests are rejected unless an explicitly local-only flag is enabled.
Idempotency	A duplicate event_id is detected before a new payment event is created.
Failure processing	Failed events are persisted, categorised from error content, scored, analysed, assigned a recovery action, and given a pending outcome record.
Prediction	Pre-payment risk endpoint and dashboard prediction details use the FastAPI artifact when available; Java rules are a safe fallback if it is unavailable.
Recovery advice	Groq can return constrained JSON advice; deterministic fallback rules choose channels for funds, cards, authentication, UPI, timeout, rate-limit, bank, gateway, repeated, customer, and unknown issues.
Payment links	For a payment_link action, calls Razorpay /v1/payment_links with Basic authentication if credentials are supplied. Otherwise records a mock link transparently.
Outcome feedback	A unique eligible payment.captured event closes a pending outcome and records recovered amount and elapsed seconds.
Dashboard	React views for payment metrics, failure distributions, recent failures, AI recovery analyses, payment details, and an ML risk form.

7. Technology stack and rationale

Technology	Why it is used here
React 19 + Vite	Fast single-page dashboard with navigation, API calls, payment detail modal, charts rendered with CSS, and the risk-prediction form.
Spring Boot 3.5 / Java 17	API layer, webhook processing, validation, recovery orchestration, and REST integration.
MySQL 8 + JPA/JdbcTemplate	Persists the audit trail and provides indexed reporting views and metrics queries.
FastAPI + Python	Hosts the model as a small local HTTP prediction service.
scikit-learn / pandas / joblib	Train, serialize, load, and score the HistGradientBoostingClassifier pipeline.
Razorpay Java SDK / REST API	Project dependency and REST call path for Razorpay payment links; webhook format is handled by the backend.
Groq OpenAI-compatible REST API	Optional constrained JSON recovery advice. The application works with deterministic rules when it is not configured.

8. Razorpay integration and webhook flow

The code expects `payment.failed` and `payment.captured`-style payloads. It retains the raw payload, normalises amount from paise to INR, maps status, and captures Razorpay identifiers, payment method, bank/wallet, error code, error description, and customer ID when available. Live links require `RAZORPAY_KEY_ID` and `RAZORPAY_KEY_SECRET`. The webhook secret is injected through environment configuration, never hard-coded.

Demo-safe wording: “This project integrates with Razorpay webhooks and has a real Payment Link API call path. In my local demonstration, credentials determine whether the link is real; the application labels the mock case honestly.”

9. ML component and prediction flow

The Python service loads `failure_predictor.joblib` and exposes `POST /predict`. It builds categorical and numeric features including transaction type, merchant category, sender/receiver attributes, bank, network, time, amount, fraud flag, weekend, peak-hour, night, and high-value flags. The training script uses a stratified 80/20 split of the included 250,000-row UPI dataset and trains a preprocessing pipeline plus `HistGradientBoostingClassifier`.

The Java backend calls this service when `ML_SERVICE_ENABLED=true`; any model-service failure falls back to `recoverflow-rules-v1`. The rules return probability, risk factors, risk level, and a confidence label. The ML endpoint returns a probability and labels its confidence as “medium”; it does not claim calibrated confidence intervals.

Model-quality disclosure for judges: the committed report records test ROC-AUC 0.490147 and average precision 0.048756. These figures do not support a claim of strong predictive performance. Present the ML service as a functioning, replaceable scoring pipeline and rules fallback—not as a production-validated predictor.

10. AI / rule-based payment recovery mechanism

First, the backend categorises the Razorpay error description/code into root causes. Then it uses Groq, if a key is configured, to create structured advice that is explicitly prohibited from predicting payment outcome. If no valid AI response is available, the rule engine creates a deterministic explanation, confidence, priority, recommended channel, recovery steps, alternatives, and retry-policy metadata.

Detected cause	Examples of implemented channel / advice
Insufficient funds, repeated failures, customer issue	Customer support: resolve the customer-side issue rather than blindly retrying.
Invalid/expired card, UPI failure, bank or gateway issue, unknown	Payment link: ask for updated details or a retry with alternatives; link creation is configuration-dependent.
Authentication failure, timeout/network, rate limit	Retry recommendation with a stored retry strategy; rate-limit advice waits longer. No automatic payment charge is performed.

11. Frontend and dashboard

- **Dashboard:** selectable period, total/successful/failed/recovered counts, recovered amount, average recovery time, risk distribution, top issue, error-code distribution, payment-method failure rate, recent failures, and up to six recovery-analysis cards.
- **Payments:** paginated and status-filtered event list. The details modal has Info, ML Prediction, and AI Analysis tabs. ML/AI tabs correctly report unavailable data for unprocessed/non-failed events.
- **ML Predictor:** form inputs to request an estimate before a payment attempt. The screen explicitly says it does not generate AI recovery advice.

12. Database architecture

```
payment_events
├─ system_insights      (failure probability, category, metadata)
├─ ai_agent_analysis    (cause, explanation, channel, strategy, reasoning)
├─ recovery_actions     (execution state, link ID/error, metadata)
│   └─ recovery_outcomes (pending/success, recovered amount, duration)
│       └─ new_payment_event_id → payment_events (captured event)
```

MySQL views: `v_recovery_pipeline` joins the journey;
`v_recovery_metrics` aggregates daily attempts, successes, amount and time.

The schema stores JSON for raw payloads and flexible strategy/metadata, uses foreign keys, timestamped records, and indexes for event IDs, payment/order IDs, status, channels, and outcomes.

13. Security and reliability

Control	Implemented evidence
Webhook authenticity	HMAC-SHA256 of raw payload using the configured webhook secret; constant-time comparison via <code>MessageDigest.isEqual</code> .
Secret handling	<code>.env.example</code> documents variables; README says never commit <code>.env</code> , database credentials, Razorpay keys, webhook secret, or Groq key.
Duplicate protection	Unique event ID and pre-save duplicate check return a duplicate response.
Safe fallback	ML HTTP errors fall back to deterministic rules; Groq missing/error also falls back to rules.
Action truthfulness	Missing payment-link credentials create a flagged mock identifier and a pending—not executed—action.
Conservative outcome matching	Capture closes an outcome only where exactly one candidate matches; ambiguous cases are left for review.
Auditability	Raw payload, reasoning, applied policy, strategy/action metadata, errors, timestamps, and outcome records are stored.

14. Testing and verified results

Verification executed while preparing this document:

Check	Verified result
<code>backend: mvn test</code>	18 tests passed, 0 failed, 0 errors. The opt-in MySQL integration test was skipped because <code>RUN_MYSQL_IT=true</code> and a local database were not supplied.
Webhook controller tests	Verified acceptance of a valid signed request (mocked service) and HTTP 401 rejection of an invalid signature.
Prediction unit tests	Verified fallback-rule risk factors, bounds, labels, consistency, and controller response structure.
Health controller test	Verified <code>/api/health</code> returns UP.
<code>frontend: npm run build</code>	Production Vite build succeeded: 37 modules transformed.

Not claimed as verified: no live Razorpay webhook, live payment link, live Groq call, browser demo with a populated MySQL database, or opt-in MySQL integration run was performed for this document. Therefore no recovery-rate, revenue, latency, or customer-impact result is claimed.

15. What makes it different

- **Recovery is cause-aware:** it maps a failure to a channel and explicit next steps rather than offering one generic retry.
- **Separation of roles:** prediction estimates risk; Groq/rules recommend recovery. The Groq prompt explicitly prevents it from inventing a payment prediction.
- **Closed-loop design:** it stores the original event, insight, advice, action, and eventual capture outcome in linked records.
- **Honest execution state:** mock versus executed payment links and pending non-integrated channels are not disguised as completed actions.
- **Operational visibility:** dashboard and reporting views connect individual failures to aggregate recovery metrics.

16. Business impact

The implemented value is operational: faster triage, consistent recovery playbooks, a path to recovery links, and a measurable recovery trail. It is designed to help a merchant focus attention on failed-payment follow-up. It does *not* establish a measured uplift in recovery rate or revenue yet; that requires live, governed operating data and evaluation.

17. Realistic end-to-end demo scenario

Scenario: A customer's ₹1,250 UPI payment for order_demo_001 is declined with a bank-decline description. A signed payment.failed webhook enters the backend. The system stores it, classifies it as a bank issue, produces a risk score, creates a cause-aware payment-link recommendation, and records a pending recovery outcome. If Razorpay credentials are configured, it requests a real Payment Link; otherwise it visibly remains a mock/pending action. When a later captured webhook with the same order and amount arrives as the only eligible candidate, the outcome becomes success and records ₹1,250 plus elapsed recovery time.

Never say: "The system automatically retries the customer's card," "we sent an SMS," or "we recovered real merchant revenue." Those are not supported by this codebase.

18. Complete 5-minute speaking script

PROBLEM SOLUTION HOW IT WORKS DEMO INNOVATION IMPACT

0:00–0:30. "A payment failure should not be the end of a customer journey. For a merchant, it is a moment of uncertainty: should we retry, wait, offer another method, send a payment link, or ask support to help? Our project, Razorpay AI Payment Recovery, turns that uncertainty into an explainable recovery path."

0:30–1:15. "A basic retry system treats every failure the same. But an expired card, an OTP failure, insufficient funds, a bank decline, and a temporary timeout need different responses. Repeating the same attempt can frustrate the customer and gives the merchant no audit trail. We built a workflow that receives the payment failure, understands its context, recommends the next action, and tracks whether a later payment is recovered."

1:15–2:15. "Here is the flow. Razorpay sends a payment webhook. Our Spring Boot backend first verifies the HMAC signature against the raw body, then stores the event idempotently in MySQL. For a failed payment, it categorises the error and requests a risk score from our FastAPI machine-learning service. If that service is not available, deterministic rules keep the workflow working. Next, a recovery advisor produces structured steps. Groq is optional; without it, our implemented rule engine chooses a transparent recommendation. For example, it avoids blind retry for insufficient funds, but recommends a delayed retry path for a timeout."

2:15–3:15. "For a payment-link recommendation, the backend has a real Razorpay Payment Link API call path. It creates the link only when credentials are configured. Otherwise it records an explicitly marked mock link as pending—so we never claim an action happened when it did not. For retry, notification, and support, this version stores the recommended action and strategy; it does not charge a customer or send an external message. Finally, a uniquely matching payment.captured webhook closes the recovery outcome and stores the recovered amount and elapsed time."

3:15–4:15. "On the dashboard, the merchant sees payment volume, failures, recovered payments, recovered amount, failure patterns, recent failures, and the latest recovery analyses. In Payments, I can open one failed event and see the original details, its risk output, and the recommended recovery steps. The ML Predictor is a separate pre-payment screen: it estimates risk, but it does not pretend to create recovery advice."

4:15–4:45. "What makes this more than a retry tool is the combination of cause-aware decisioning, clear separation between prediction and advice, honest execution status, and a closed audit trail from webhook to outcome. The project is also resilient: signed webhooks, duplicate detection, rules fallback for both the model and the AI advisor, and conservative matching when a capture could be ambiguous."

4:45–5:00. "The business value is simple: help merchants move from seeing a failed payment to taking a defensible next step and measuring the result. Razorpay AI Payment Recovery does not promise magic recovery—it makes recovery intelligent, traceable, and ready to improve with real operational data. Thank you."

19–20. Exact 5-minute video / screen sequence

TIME	WHAT I SAY	WHAT I SHOW
0:00–0:30	Use the opening from the script: “A payment failure should not be the end...” State the one-line pitch.	Title slide or the dashboard landing screen. Keep the project name large and clean; do not show terminal windows.
0:30–1:15	Explain that generic retries do not fit different causes; introduce cause-aware recovery and audit trail.	Show the workflow diagram from this PDF or a simple slide made from it. Highlight: failed webhook → insight → advice → action → outcome.
1:15–2:15	Explain signature verification, idempotent storage, ML score/rules fallback, and AI/rules advice.	Show the workflow diagram while pointing one block at a time. Then show the ML Predictor form briefly. Do not claim the displayed probability is accurate in production.
2:15–3:15	Explain Payment Link behavior and the capture-feedback loop. State the honest limitation of pending retry/notification/support channels.	Show the Payments view and open a failed payment. In AI Analysis, point to cause, recommended action, steps, alternatives, priority. If no local data is available, use the supplied sample webhook JSON in the editor and narrate the expected processed path; do not fabricate a dashboard record.
3:15–4:15	Walk through merchant visibility: dashboard, failure patterns, payment details, risk, AI advice.	Show Dashboard metrics and AI Recovery Dashboard. Then Payments → a failed record → ML Prediction tab → AI Analysis tab. Only show populated results that actually exist locally.
4:15–4:45	State the differentiators: cause-aware, prediction/advice separation, honest state, closed loop, security/fallbacks.	Return to the architecture diagram. Overlay or verbally enumerate the five differentiators; avoid clutter.
4:45–5:00	Deliver the closing line slowly and confidently.	Return to title slide with the one-line pitch and “Intelligent • Traceable • Cause-aware”.

Recording order before the video: start ML service, backend with the `mysql` profile and a configured local MySQL database, then frontend. Use Razorpay Test Mode only if valid test credentials and webhook configuration are already available. The project contains sample failed/captured webhook payloads and a local signing smoke-test script. If those prerequisites are not available, demonstrate the UI and code-backed workflow without implying a live end-to-end run.

21. Fifteen likely judge questions and strong answers

Question	Answer
Is this just a retry system?	No. It classifies failure context and selects a recovery channel and steps. It distinguishes, for example, timeout from expired card or insufficient funds, and records the entire decision and outcome trail.
Does it automatically retry a charge?	No. The current version records retry advice and strategy but does not re-charge the customer. That is deliberate scope control.
What is actually AI here?	The implementation has an optional Groq structured recovery advisor. It is constrained to advise on recovery, not to predict payment outcomes. It falls back to deterministic rules for reliability.
What does the ML model predict?	Failure probability from transaction features. The same backend has a rule-based fallback if the local ML service cannot respond. Razorpay AI Payment Recovery • Pitch documentation

How accurate is the model?	The committed report has weak holdout discrimination—ROC-AUC 0.490147. I would not claim production accuracy. The functioning pipeline is ready for better data, calibration, and governed evaluation.
How do you secure webhooks?	HMAC-SHA256 on the exact raw body, configured secret, constant-time comparison, and rejection of invalid signatures. Unsigned acceptance is an explicit local-only setting.
How do you avoid duplicate events?	The Razorpay event ID is unique in the schema and checked before saving. Duplicates return a duplicate status rather than rerunning recovery.
Does it send SMS or email?	No. Those channels are stored as pending recommendations. The code does not claim provider delivery.
Does it create a real Razorpay link?	It has a real REST call path to Razorpay Payment Links when test/live credentials are configured. Without credentials it creates a labelled mock link and leaves it pending.
How is recovery success determined?	On a later captured webhook, it matches the expected replacement payment ID or same order plus amount. It closes the outcome only when exactly one pending candidate matches.
Why use both ML and rules?	ML provides a service boundary for probability scoring; rules give predictable behavior, explainability, and continuity if the model service is unavailable.
Why separate AI advice from prediction?	It avoids asking a generative model to invent financial probabilities. The advisor receives the existing score and error context only to choose recovery guidance.
What does MySQL store?	Raw event, insight, analysis, action, and outcome records linked by foreign keys, plus audit metadata and reporting views.
What was tested?	18 backend tests passed and the frontend production build passed. The MySQL integration test is present but was not run without its explicit database prerequisite.
What is the next validation step?	Run the opt-in MySQL integration test and Razorpay Test Mode workflow, then evaluate recovery strategies on real, consented operational data before making business claims.

22. Three powerful closing statements

1. “We do not treat failed payments as dead ends; we turn them into traceable recovery decisions.”
2. “The intelligence is not a blind retry—it is choosing the next best merchant action from the actual failure context.”
3. “Razorpay AI Payment Recovery makes the path from failure to follow-up visible, explainable, and measurable.”

23. One-page final cheat sheet

Opening: “A payment failure should not be the end of a customer journey. We turn it into an explainable recovery path.”

Remember	Say it this way
Core flow	Signed webhook → MySQL event → score + category → Groq/rules recovery advice → recorded action → captured webhook closes outcome.
Why it matters	Merchants get an actionable next step and an audit trail, not just a failure notification.
More than retry	Different failures trigger different recommendations; repeated retries are not automatically treated as the answer.
ML truth	Functional local scoring pipeline with rules fallback. Do not claim strong accuracy; the committed model report is weak.
AI truth	Optional Groq advice is constrained to recovery strategy, not outcome prediction; deterministic rules always back it up.
Razorpay truth	Signature verification and Payment Link API path are implemented. Live links need credentials; otherwise mock/pending is explicit.
Automation truth	Failure processing and action recording are event-driven. No automatic charge retry, SMS, email, or support delivery is implemented.
Proof	18 backend tests passed; frontend production build passed. MySQL integration test exists but was opt-in/skipped.
Close	“We make recovery intelligent, traceable, and cause-aware—without pretending that a recommendation is an action already taken.”

Pitch posture

- Speak slowly at the first and last 15 seconds. Let the problem land.
- When showing a feature, name the exact outcome it provides: context, recommendation, audit trail, or visibility.
- If asked about a missing integration, answer directly, then point to the stored pending action and honest state handling.
- Never invent results. Credibility is a differentiator in payment systems.

Document basis: repository files reviewed across backend, frontend, ML, database schema/docs, sample webhooks, environment templates, and test suite. Generated as project documentation; no project source code was modified.